

DP-LMI package

User Manual

Version v0.1.0

July 2026

Abstract

This manual describes the current public behavior of the DP-LMI package. The present implementation uses `dpbase` as the backend parent for cell-local Bernstein coefficient storage, provides `dpmat` for known finite real matrix data on tensor grids, and provides an initial `dpvar` layer for continuous degree-1 YALMIP-backed decision expressions plus cell-local rate-weighted derivatives through `rhodiff`. It also provides the first `dplmi` layer: `dpvar` comparisons assemble direct coefficient-wise YALMIP constraints, and `toYalmip` exposes them to `optimize`. Relaxation-lemma and PolyA workflows remain reserved future extensions.

Contents

1	Installation And Verification	4
1.1	Add The Package To The MATLAB Path	4
1.2	Run The Current Test Suite	4
2	Quick Start	5
2.1	A Known Matrix On A Scalar Grid	5
2.2	A Known Matrix On A Tensor Grid	5
2.3	A Function-Backed Known Matrix	6
2.4	A Continuous Decision Variable	6
3	Bernstein And Grid Background	8
3.1	One Cell, Local Coordinates	8
3.2	Quadratic And Higher-Degree Bases	8
3.3	Products And Degree Growth	9
3.4	Tensor-Product Labels	10
3.5	Data Layout Seen From MATLAB	11
4	dpbase Backend Architecture	13
4.1	Purpose	13
4.2	Backend Constructor Contract	13
4.3	Backend Inputs	13
4.4	Backend Metadata Options	14
4.5	Inherited Inspection Properties	14
4.6	Inherited Inspection Methods	15
4.7	Backend Local-Storage Example	15
4.8	Backend Rate Metadata	16
4.9	Backend Validation And Error Boundaries	16
5	dpmat Reference	17
5.1	Purpose	17
5.2	Syntax	17
5.3	Source Modes	18
5.4	Evaluation	18
5.5	Coefficient Algebra	19
5.6	Matrix Operations And Indexing	19
5.7	Shape And Structural Methods	20
5.8	Display And Plotting	21
5.9	Validation Boundaries	22
6	dpvar Reference	24
6.1	Purpose	24
6.2	Syntax	24
6.3	Construction And Continuity	24
6.4	Rate Metadata	25
6.5	Rate-Weighted Derivatives With <code>rhodiff</code>	26

6.6	Affine Algebra And Products	27
6.7	Matrix Operations And Indexing	27
6.8	Validation Boundaries	28
7	dplmi Assembly And Solver-Facing Use	29
7.1	Purpose	29
7.2	Syntax	29
7.3	Direct Coefficient Constraints	29
7.4	Rate-Vertex Assembly From <code>rhodiff</code>	30
7.5	A Lyapunov-Style Solver Smoke Test	30
7.6	A Block DP-LMI With A Performance Variable	31
7.7	Validation Boundaries	32
8	Current Limits	33
9	Style References Used For This Manual	34

1

Installation And Verification

1.1 Add The Package To The MATLAB Path

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

```
projectRoot = "path/to/Differential Parameter-Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local development from the repository root, the shorter form is enough:

```
addpath(pwd)
```

1.2 Run The Current Test Suite

The current verification entry point runs the tests for `+helper`, `@dpbase`, `@dpmat`, `@dpvar`, and `@dplmi`. The `@dpvar` and `@dplmi` tests require YALMIP to be available on the MATLAB path. Solver smoke tests use installed YALMIP solvers; the MOSEK example is skipped unless MOSEK is available.

```
results = tests.run_all;
table(results)
```

Expected result: all currently implemented tests pass. The suite covers helper validation, `dpbase` construction, grid metadata, local storage, label ordering, rate-bound validation, and current `dpmat` construction, evaluation, algebra, display, plotting, matrix operations, indexing, assignment, structural transforms, and error boundaries. It also covers the current `dpvar` constructor, continuity-by-shared-coefficients behavior, affine algebra, mixed known/decision products, matrix operations, indexing, assignment, `rhodiff` rate-vertex derivative storage, and validation boundaries. The `dplmi` tests cover comparison overloads, coefficient-wise constraint assembly, rate-vertex enumeration, `toYalmip`, reserved relaxation/Polya option errors, and small solver-facing smoke examples. Full relaxation-lemma assembly, Polya assembly, theorem residual margins, strictness margins, and solver diagnostics are not implemented yet.

2

Quick Start

Most users should start with `dpmat` for known parameter-dependent data and `dpvar` for YALMIP-backed decision expressions. Both classes extend the shared `dpbase` backend, so they expose common grid, cell, label, and coefficient-inspection methods; ordinary modeling code should not need to construct raw `dpbase` objects.

2.1 A Known Matrix On A Scalar Grid

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);

A.MatrixSize
A.Degree
A.SourceSummary
A.evaluate(0.25)
C = A + 5;
C.coeffs(1)
A.cells()
A.lbls()
```

Expected structural outputs:

```
MatrixSize =
    1     1

Degree =
    1

SourceSummary =
    "coefficient-backed"

A.evaluate(0.25) -> 2.5
C.coeffs(1)      -> {7, 9}

cells() =
    1

lbls() =
    0
    1
```

The expression `A + 5` returns another coefficient-backed `dpmat` on the same grid. The `cells`, `lbls`, and `coeffs` methods come from the shared `dpbase` backend, but the user-facing object remains a `dpmat`.

2.2 A Known Matrix On A Tensor Grid

```
A = dpmat({[0 1], [10 20]}, {1, 3; 5, 7}, Degree=1);

A.GridInfo.Points
A.cells()
A.lbls()
A.coeffs([1 1])
```

Expected structural outputs:

```
GridInfo.Points =
    0    10
    0    20
    1    10
    1    20

cells() =
    1    1

lbls() =
    0    0
    0    1
    1    0
    1    1

A.coeffs([1 1]) -> {1, 3, 5, 7}
```

The first parameter index varies more slowly than the second. This row order is used consistently for physical cells and local Bernstein labels across `dpmat` and `dpvar`, because both inherit the same backend storage contract.

2.3 A Function-Backed Known Matrix

```
F = dpmat({[0 pi]}, @(rho) sin(rho));

F.SourceSummary
F.evaluate(pi/2)
F.coeffs(1)
```

Expected outputs:

```
SourceSummary =
    "function"

F.evaluate(pi/2) -> 1
F.coeffs(1)      -> {0, 0}
```

Function-backed `dpmat` objects without an explicit `Degree` keep the exact function handle for evaluation. The inherited local coefficients are placeholder zeros used only to satisfy shared `dpbase` storage; they are not Bernstein evidence and cannot enter coefficient algebra.

2.4 A Continuous Decision Variable

```

P = dpvar(2, {[0 1 2]}, "symmetric");

P.MatrixSize
P.Degree
P.IsContinuous
P.ContainsDecision
left = P.coeffs(1);
right = P.coeffs(2);
isequal(getvariables(left{2}), getvariables(right{1}))

```

Expected structural outputs:

```

MatrixSize =
    2    2

Degree =
    1

IsContinuous    -> true
ContainsDecision-> true
shared boundary -> true

```

`dpvar` creates YALMIP decision coefficients on the global grid nodes and reuses the same coefficient handles in adjacent physical cells. Continuity is therefore built into storage; no equality LMI is added by the constructor.

3

Bernstein And Grid Background

3.1 One Cell, Local Coordinates

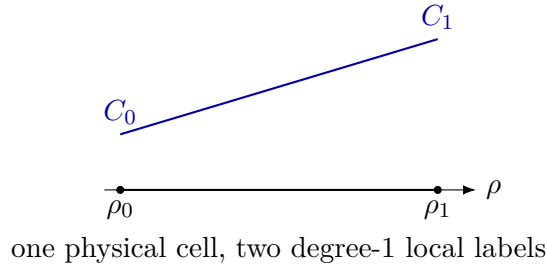
On one physical interval $[\rho_0, \rho_1]$, the package uses the normalized coordinate

$$\alpha = \frac{\rho_1 - \rho}{\rho_1 - \rho_0}, \quad 0 \leq \alpha \leq 1.$$

A degree- m Bernstein polynomial is written as

$$P(\alpha) = \sum_{i=0}^m C_i B_i^m(\alpha), \quad B_i^m(\alpha) = \binom{m}{i} \alpha^{m-i} (1-\alpha)^i.$$

For matrix-valued data, each coefficient C_i is a matrix with the same payload size. **dpbase** stores those matrices as one flat coefficient cell for each physical grid cell.



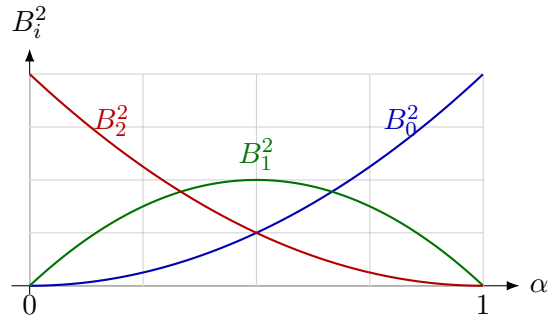
Degree one is the line-segment case. It is useful intuition for piecewise-linear **dpmat** data and for the current degree-1 **dpvar** constructor, but Bernstein storage is not limited to splines.

3.2 Quadratic And Higher-Degree Bases

For degree two, one cell has three local labels 0, 1, 2:

$$P(\alpha) = C_0 \alpha^2 + 2C_1 \alpha(1-\alpha) + C_2 (1-\alpha)^2.$$

The middle coefficient C_1 controls the interior Bernstein basis function, not an extra physical grid node. The physical interval still has two endpoints; the polynomial degree controls how many local coefficient labels live inside that cell.

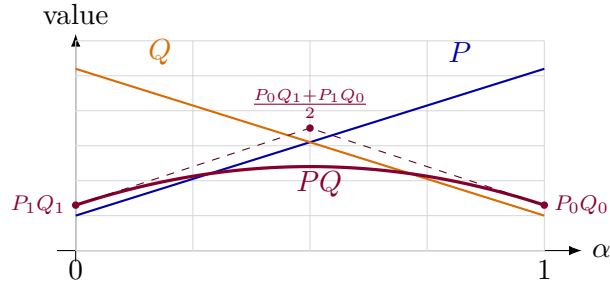


The same pattern extends to any degree m . A cubic cell stores four coefficient labels 0, 1, 2, 3, a quartic cell stores five, and so on. The labels are local to the current physical cell; adjacent cells have their own local coefficient cells even when continuous objects share boundary handles.

3.3 Products And Degree Growth

Multiplication is the main reason the package treats Bernstein labels as coefficient data rather than as samples. On one cell, multiplying two degree-1 factors produces a degree-2 polynomial:

$$\begin{aligned} P(\alpha) &= \alpha P_0 + (1 - \alpha) P_1, \\ Q(\alpha) &= \alpha Q_0 + (1 - \alpha) Q_1, \\ P(\alpha)Q(\alpha) &= B_0^2 P_0 Q_0 + B_1^2 \frac{P_0 Q_1 + P_1 Q_0}{2} + B_2^2 P_1 Q_1. \end{aligned}$$



The dashed purple control polygon is not another sampled curve. It marks the three degree-2 Bernstein coefficients of the product. The middle coefficient is higher than the curve value at $\alpha = 1/2$ because the basis function $B_1^2(\alpha) = 2\alpha(1 - \alpha)$ already carries a factor of two.

For general scalar degrees n and m ,

$$P(\alpha) = \sum_i P_i B_i^n(\alpha), \quad Q(\alpha) = \sum_j Q_j B_j^m(\alpha),$$

and

$$P(\alpha)Q(\alpha) = \sum_{k=0}^{n+m} R_k B_k^{n+m}(\alpha),$$

where

$$R_k = \sum_{i+j=k} P_i Q_j \frac{\binom{n}{i} \binom{m}{j}}{\binom{n+m}{k}}.$$

This is coefficient convolution with Bernstein binomial normalization. An equivalent way to see the same rule, matching the source note `DLMI with Bernstein Polynomial.pdf`, is to first absorb the binomial weights into the coefficient vectors:

$$\hat{P}_i = \binom{n}{i} P_i, \quad \hat{Q}_j = \binom{m}{j} Q_j.$$

Then the weighted product coefficients are the ordinary discrete convolution

$$\hat{R}_k = (\hat{P} * \hat{Q})_k = \sum_{i+j=k} \hat{P}_i \hat{Q}_j,$$

and the Bernstein coefficients used by the package are recovered by dividing back by the degree- $n+m$ binomial weight:

$$R_k = \frac{\hat{R}_k}{\binom{n+m}{k}}.$$

For the linear case, the unnormalized convolution is simply

$$[P_0, P_1] * [Q_0, Q_1] = [P_0Q_0, P_0Q_1 + P_1Q_0, P_1Q_1],$$

and the middle Bernstein coefficient is divided by $\binom{2}{1} = 2$. For example, if P has degree two and Q has degree three, then the degree-five coefficient with label $k = 2$ is

$$R_2 = \frac{3}{10}P_0Q_2 + \frac{6}{10}P_1Q_1 + \frac{1}{10}P_2Q_0.$$

This is the binomial-normalized convolution used by coefficient-backed `dpmat` multiplication and by mixed known/decision products that remain affine in YALMIP variables.

For tensor-product Bernstein polynomials, the same convolution happens over multi-indices. If p , q , and k are local labels, then product terms contribute only when $p+q = k$, and the one-dimensional binomial ratios multiply axis by axis:

$$R_k = \sum_{p+q=k} P_p Q_q \prod_{r=1}^{\ell} \frac{\binom{n}{p_r} \binom{m}{q_r}}{\binom{n+m}{k_r}}.$$

Thus two multilinear 2×2 coefficient arrays in two parameters become one 3×3 degree-2 coefficient array. This is the tensor version of the same coefficient convolution, not a multiplication of physical grid samples.

3.4 Tensor-Product Labels

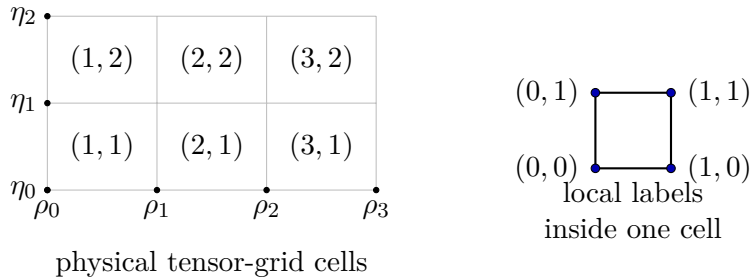
For ℓ parameter dimensions and scalar degree m , each local label is an ℓ -tuple in $\{0, \dots, m\}^\ell$. The number of local coefficients per physical cell is

$$(m+1)^\ell.$$

For example, $\ell = 2$ and $m = 1$ gives labels

$$(0, 0), (0, 1), (1, 0), (1, 1),$$

which is exactly the order returned by `obj.lbls()`.



Tensor-product multiplication adds labels component-wise. In two dimensions, a label (i_1, i_2) from degree n and a label (j_1, j_2) from degree m contribute to result label

$$(k_1, k_2) = (i_1 + j_1, i_2 + j_2),$$

with a scale factor that is the product of the one-dimensional binomial ratios. For three parameters, the same rule applies to triples such as $(i_1, i_2, i_3) + (j_1, j_2, j_3)$. This is why a degree-1 object over a three-dimensional parameter grid has $2^3 = 8$ local coefficient labels per physical cell.

3.5 Data Layout Seen From MATLAB

The physical grid and the local Bernstein labels are separate. A two-parameter `dpmat` with a degree-1 global coefficient grid stores four local coefficients per physical cell:

```
A = dpmat({[0 1], [10 20]}, {1, 3; 5, 7}, Degree=1);

A.npar()
A.ncell()
A.ncoeff()
A.lbls()
A.coeffs([1 1])
A.evaluate([0.25, 12.5])
```

Expected structural outputs:

```
npar()    -> 2
ncell()   -> 1
ncoeff()  -> 4

lbls() =
    0    0
    0    1
    1    0
    1    1

coeffs([1 1]) -> {1, 3, 5, 7}
evaluate([0.25 12.5]) -> 2.5
```

A degree-2 scalar cell stores three coefficients and evaluates with the quadratic basis shown above.

```
A = dpmat({[0 1]}, {1, 2, 4}, Degree=2);

A.Degree
A.ncoeff()
A.evaluate(0.5)
```

Expected output:

```
Degree    -> 2
ncoeff()  -> 3
evaluate(0.5) -> 2.25
```

The current `dpvar` constructor is degree one, but it uses the same cell-local label ordering and tensor-grid storage.

```
P = dpvar(1, {[0 1], [10 20]});
```

```
P.Degree
```

```
P.ncoeff()
```

```
P.lbls()
```

Expected structural output:

```
Degree    -> 1
```

```
ncoeff()  -> 4
```

```
lbls() =
```

```
    0    0
```

```
    0    1
```

```
    1    0
```

```
    1    1
```

4

dpbase Backend Architecture

4.1 Purpose

`dpbase` is the shared backend parent class for gridded DP-LMI objects. `dpmat` and `dpvar` extend from it: `dpmat` adds known numeric data and exact evaluation, while `dpvar` adds YALMIP-backed decision coefficients and affine expression behavior. Ordinary user workflows should construct `dpmat` and `dpvar`, not raw `dpbase` objects.

The purpose of this section is architectural. It explains why `dpmat` and `dpvar` expose the same grid, cell, local-label, coefficient, and metadata inspection behavior. The parent class stores tensor-grid metadata, matrix payload size, Bernstein degree, nested physical-cell coefficient data, and rate-dependence metadata.

4.2 Backend Constructor Contract

The raw constructor is mainly useful for implementation, tests, and reading the inherited storage contract. It is not the recommended modeling entry point.

```
obj = dpbase(gridVectors, matrixSize, degree)
obj = dpbase(gridVectors, matrixSize, degree, localValues)
obj = dpbase(gridVectors, matrixSize, degree, localValues, Name=Value)
```

4.3 Backend Inputs

Input	Description
<code>gridVectors</code>	Nonempty cell array of grid node vectors. Each vector must be finite, real, strictly increasing, numeric, and contain at least two nodes.
<code>matrixSize</code>	1x2 positive integer vector giving the row and column size of every coefficient payload.
<code>degree</code>	Nonnegative integer scalar Bernstein degree.
<code>localValues</code>	Optional nested cell payload. If omitted or empty, <code>dpbase</code> creates zero numeric coefficient matrices for every local coefficient of every physical cell.

4.4 Backend Metadata Options

Option	Description
<code>IsContinuous</code>	Logical metadata flag. <code>dpbase</code> records the flag but does not enforce boundary sharing itself. Subclasses such as <code>dpvar</code> own continuity construction.
<code>ContainsDecision</code>	Logical metadata flag used by decision-variable subclasses such as <code>dpvar</code> .
<code>HasRateDependence</code>	Logical metadata flag. If true, nonempty <code>RateBounds</code> must be supplied.
<code>RateBounds</code>	Empty, or an $\ell \times 2$ finite real matrix with lower bounds in column one and upper bounds in column two. The number of rows must match the number of parameter dimensions.
<code>SourceSummary</code>	String metadata describing the coefficient source. The default is "coefficient-backed".

4.5 Inherited Inspection Properties

All public properties have private set access. Users normally inspect these properties through `dpmat` or `dpvar` objects; they cannot assign to them after construction.

Property	Meaning
<code>GridInfo</code>	Struct with fields <code>Vectors</code> , <code>Points</code> , <code>Bounds</code> , and <code>NumNodes</code> .
<code>MatrixSize</code>	Matrix payload size stored as a 1x2 vector.
<code>Degree</code>	Bernstein degree.
<code>LocalValues</code>	Nested physical-cell storage. A tensor grid uses nested brace access, for example <code>LocalValues{i1}</code> followed by <code>{i2}</code> and later dimensions; each leaf stores a flat coefficient cell.
<code>IsContinuous</code>	Continuity metadata flag.
<code>ContainsDecision</code>	Decision-dependence metadata flag.
<code>HasRateDependence</code>	True when rate metadata or rate-affine payloads are present.
<code>RateBounds</code>	Empty or an $\ell \times 2$ finite real rate-bound table.
<code>SourceSummary</code>	String metadata for the coefficient source.

4.6 Inherited Inspection Methods

Method	Behavior
<code>size(obj)</code>	Return the matrix payload size. <code>size(obj,dim)</code> follows ordinary matrix behavior for dimensions beyond two by returning one.
<code>npar(obj)</code>	Return the number of parameter dimensions.
<code>ncell(obj)</code>	Return the number of physical tensor-grid cells.
<code>ncoeff(obj)</code>	Return the number of local Bernstein coefficients per cell.
<code>cells(obj)</code>	Enumerate physical-cell subscripts in flat row order.
<code>lbls(obj)</code>	Enumerate local Bernstein labels in flat row order.
<code>coeffs(obj,cellSubs)</code>	Return the flat coefficient cell for one physical cell. <code>cellSubs</code> can be a numeric row vector or a cell array with one physical-cell index per parameter.

4.7 Backend Local-Storage Example

The examples below show the parent storage contract for readers who need to understand inherited behavior or implementation tests. They are not the recommended starting point for modeling; use `dpmat` for known data and `dpvar` for decision expressions.

For a one-dimensional grid with two physical cells and degree two, each physical cell stores three local coefficients.

```
localValues = {{11, 12, 13}, {21, 22, 23}};
obj = dpbase({[0 1 2]}, [1 1], 2, localValues);

obj.coeffs(2)
obj.coeffs({1})
```

Expected output:

```
obj.coeffs(2) -> {21, 22, 23}
obj.coeffs({1})-> {11, 12, 13}
```

For a tensor grid, physical cells are nested one parameter dimension at a time. The storage contract is `LocalValues{i1}{i2}...`, not `LocalValues{i1,i2,...}`.

```
mkCoeff = @(offset) {offset+1, offset+2, offset+3, offset+4};
localValues = {
    {mkCoeff(110), mkCoeff(120)}, ...
    {mkCoeff(210), mkCoeff(220)}
};

obj = dpbase({[0 1 2], [10 20 30]}, [1 1], 1, localValues);
obj.coeffs([2 1])
```

Expected output:

```
ans =
1x4 cell array
    {[211]}    {[212]}    {[213]}    {[214]}
```

4.8 Backend Rate Metadata

`dpbase` validates rate bounds and rate-affine coefficient payloads, but it does not enumerate rate vertices or assemble LMIs. Public derivative behavior belongs to `dpvar/rhodiff`; solver-facing finite LMI assembly belongs to `dplmi`.

```
obj = dpbase([0 1], [10 20]), [1 1], 0, [], ...
    RateBounds=[-1 1; -2 2]);

obj.HasRateDependence
obj.RateBounds
obj.coeffs([1 1])
```

Expected result: `HasRateDependence` is true, `RateBounds` is the given 2x2 table, and the single local coefficient remains the numeric zero payload.

Rate-affine payloads use a scalar struct with fields `Constant` and `Rate`. The `Rate` field must be a cell array with one matrix per parameter dimension.

```
payload = struct("Constant", eye(2), "Rate", {{ones(2), 2*ones(2)}});
localValues = {{payload}};
obj = dpbase([0 1], [10 20]), [2 2], 0, localValues, ...
    RateBounds=[-1 1; -2 2]);
```

The stored coefficient represents the compact form

$$C(\dot{\rho}) = C_0 + C_1\dot{\rho}_1 + C_2\dot{\rho}_2,$$

but `dpbase` only stores and validates this payload. It does not solve or test any inequality involving $\dot{\rho}$.

4.9 Backend Validation And Error Boundaries

The parent constructor rejects malformed storage inputs early and uses domain-specific error identifiers. The following examples are expected to error:

```
dpbase([0 1], [1 1], 0) % dpbase:InvalidGrid
dpbase([0 1 1], [1 1], 0) % dpbase:InvalidGridVector
dpbase([0 1], [1 0], 0) % dpbase:InvalidMatrixSize
dpbase([0 1], [1 1], -1) % dpbase:InvalidDegree
dpbase([0 1], [1 1], 0, [], ...
    HasRateDependence=true) % dpbase:InvalidRateBounds
```

Ordinary coefficient payloads at the `dpbase` layer must be finite, real, numeric matrices or real two-dimensional `sdpvar` matrices matching `matrixSize`. The parent class validates storage shape only; user-facing symbolic algebra is defined by `dpvar`.

5

dpmat Reference

5.1 Purpose

`dpmat` represents known finite real matrix data on a parameter grid. It inherits `dpbase` storage and adds source handling, exact function evaluation, coefficient-backed algebra, matrix structural operations, and two-dimensional block indexing for known data.

5.2 Syntax

```
A = dpmat(gridVectors, source)
A = dpmat(gridVectors, source, Degree=m)
val = A.evaluate(point)
val = evaluate(A, point)
C = A + B
C = A - B
C = -A
C = +A
C = A * B
C = A * M
C = M * A
T = A.'
H = A'
C = [A, B]
C = [A; B]
C = cat(1, A, B)
C = cat(2, A, B)
B = A(rows, cols)
A(rows, cols) = rhs
d = length(A)
d = height(A)
d = width(A)
d = numel(A)
d = ndims(A)
B = squeeze(A)
tf = isequal(A, B)
d = diag(A)
B = diag(A, k)
B = reshape(A, m, n)
B = reshape(A, [], n)
v = vec(A)
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
B = blkdiag(A, M)
disp(A)
display(A)
```

```
h = plot(A)
h = plot(A, dims, SamplesPerCell=n, Name, Value)
```

`source` can be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested `LocalValues` in the `dpbase` contract.

5.3 Source Modes

Mode	Behavior
Function handle, no <code>Degree</code>	Stores the exact <code>FunctionHandle</code> , probes the lower grid point only to infer matrix size, sets <code>Degree=1</code> , and leaves inherited coefficients as placeholder zeros. Function-only objects use <code>SourceSummary="function"</code> and cannot enter coefficient algebra.
Function handle with <code>Degree</code>	Validates that the handle is representable as local Bernstein data of that degree, populates <code>LocalValues</code> , keeps the exact <code>FunctionHandle</code> , and uses <code>SourceSummary="function-bernstein"</code> .
Global cell grid	Converts overlapping global Bernstein coefficient grids into flat local coefficient cells. The public helper <code>helper.cell2bern</code> exposes this same conversion for inspection. For degree m , the global cell array size must be $(n_{\text{cell}}m) + 1$ along each parameter dimension.
Nested <code>LocalValues</code>	Passes explicit local coefficients through the <code>dpbase</code> nested-cell contract. The degree is inferred from the flat coefficient count $(m + 1)^\ell$ when it is not supplied explicitly.

All current `dpmat` objects are continuous known data. Their fixed metadata is:

- `IsContinuous=true`
- `ContainsDecision=false`
- `HasRateDependence=false`

Constructor options that try to override those metadata fields are rejected.

5.4 Evaluation

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
A.evaluate(0.75)

B = dpmat({[0 1], [10 20]}, {1, 3; 5, 7}, Degree=1);
B.evaluate([0.25, 12.5])

F = dpmat({[0 pi]}, @(rho) sin(rho));
F.evaluate(pi/2)
```

Expected outputs:

```
A.evaluate(0.75)      -> 3.5
B.evaluate([0.25 12.5]) -> 2.5
F.evaluate(pi/2)      -> 1
```

Coefficient-backed objects evaluate by Bernstein interpolation on the active physical cell. Function-backed objects evaluate through the retained exact handle. Evaluation points must be finite, real, have one entry per parameter dimension, and lie inside the grid bounds.

5.5 Coefficient Algebra

`dpmat` supports coefficient-backed `+`, `-`, unary `-`, and matrix `*`. Numeric scalars and compatible numeric matrices promote to constant `dpmat` operands.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {10, 20, 30}, Degree=2);
C = A + B;
```

```
C.Degree
C.coeffs(1)
```

Expected output:

```
C.Degree    -> 2
C.coeffs(1)-> {11, 21.5, 32}
```

Multiplication raises Bernstein degree through binomial-normalized local coefficient convolution.

```
A = dpmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
B = dpmat({[0 1]}, {[5; 6], [7; 8]}, Degree=1);
K = A * B;
```

```
K.Degree
size(K)
K.coeffs(1)
```

Expected output:

```
K.Degree    -> 2
size(K)     -> [1 1]
K.coeffs(1)-> {17, 31, 53}
```

Same-bound mixed scalar grids are aligned on a common refinement before coefficient algebra. Mixed physical bounds are rejected. Function-only `dpmat` objects are rejected by coefficient algebra until exact function-handle algebra is designed.

5.6 Matrix Operations And Indexing

`dpmat` supports transpose, conjugate transpose for real data, horizontal and vertical concatenation, `cat` along dimensions 1 and 2, two-dimensional matrix slicing, block assignment, matrix-payload shape inspection, and common structural transforms. Row and column subscripts support `:`, positive integer vectors, full-length logical vectors, and `end`.

```
A = dpmat({[0 1]}, {
    [1 2 3; 4 5 6], ...
    [10 20 30; 40 50 60]
}, Degree=1);

lastCol = A(:, end);
firstRow = A([true false], :);
```

```
size(lastCol)
lastCol.coeffs(1)
```

Expected output:

```
size(lastCol)      -> [2 1]
lastCol.coeffs(1)-> {[3; 6], [30; 60]}
```

Assignment accepts numeric constants and compatible **dpmat** blocks. It does not support deletion assignment, matrix growth, unsupported dimensions, or coefficient operations on function-only objects.

```
A = dpmat({[0 1]}, {zeros(2), 2*ones(2)}, Degree=1);
A(:, 2) = 5;
A.coeffs(1)
```

Expected output:

```
A.coeffs(1)-> {[0 5; 0 5], [2 5; 2 5]}
```

5.7 Shape And Structural Methods

Shape methods report the size of each matrix payload, not the number of physical cells or coefficients.

```
A = dpmat({[0 1]}, {zeros(2, 3), ones(2, 3)}, Degree=1);

length(A)
height(A)
width(A)
numel(A)
ndims(A)
squeeze(A)
```

Expected output:

```
length(A) -> 3
height(A) -> 2
width(A) -> 3
numel(A) -> 6
ndims(A) -> 2
squeeze(A)-> A
```

Coefficient-backed structural transforms map the same MATLAB operation over every local coefficient payload.

```
A = dpmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);

vec(A).coeffs(1)
diag(A).coeffs(1)
reshape(A, [1 4]).coeffs(1)
tril(A).coeffs(1)
triu(A, 1).coeffs(1)
```

Expected output:

```
vec(A).coeffs(1)      -> {[1; 3; 2; 4], [10; 30; 20; 40]}
diag(A).coeffs(1)     -> {[1; 4], [10; 40]}
reshape(A,[1 4]).coeffs(1) -> {[1 3 2 4], [10 30 20 40]}
tril(A).coeffs(1)     -> {[1 0; 3 4], [10 0; 30 40]}
triu(A,1).coeffs(1)   -> {[0 2; 0 0], [0 20; 0 0]}
```

For vector payloads, `diag(A,k)` constructs an offset diagonal matrix. `reshape` also accepts one inferred empty dimension.

```
A = dpmat({[0 1]}, {[1; 2; 3], [4; 5; 6]}, Degree=1);

diag(A, 1).coeffs(1)
reshape(A, [], 1).coeffs(1)
```

Expected output:

```
diag(A,1).coeffs(1)      -> {[0 1 0 0; 0 0 2 0; 0 0 0 3; 0 0 0 0],
                             [0 4 0 0; 0 0 5 0; 0 0 0 6; 0 0 0 0]}
reshape(A,[],1).coeffs(1)-> {[1; 2; 3], [4; 5; 6]}
```

`blkdiag` accepts coefficient-backed `dpmat` operands and compatible numeric blocks. It aligns same-bound grids on a common refinement and elevates degree as needed.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 0.5 1]}, {10, 20, 30}, Degree=1);
C = blkdiag(A, 5, B);

size(C)
C.GridInfo.Vectors{1}
```

Expected output:

```
size(C)      -> [3 3]
C.GridInfo.Vectors -> [0 0.5 1]
```

5.8 Display And Plotting

`disp(A)` prints a compact one-line summary. Plain command-window display or `display(A)` prints matrix size, parameter grid metadata, degree, physical-cell count, coefficients per cell, continuity, source, and function-handle status.

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
```

Expected output contains:

```
dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed
```

`plot(A)` samples `evaluate` over one or two selected parameter dimensions and labels each matrix entry in the legend. For objects with more than two parameter dimensions, unselected dimensions are fixed at their lower grid bounds.

```
A = dpmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4);
numel(h)
```

Expected result: `numel(h)` is 2, one plotted line for each matrix entry. Legends use LaTeX labels such as $A_{1,1}$ and the ρ -axis label uses LaTeX formatting.

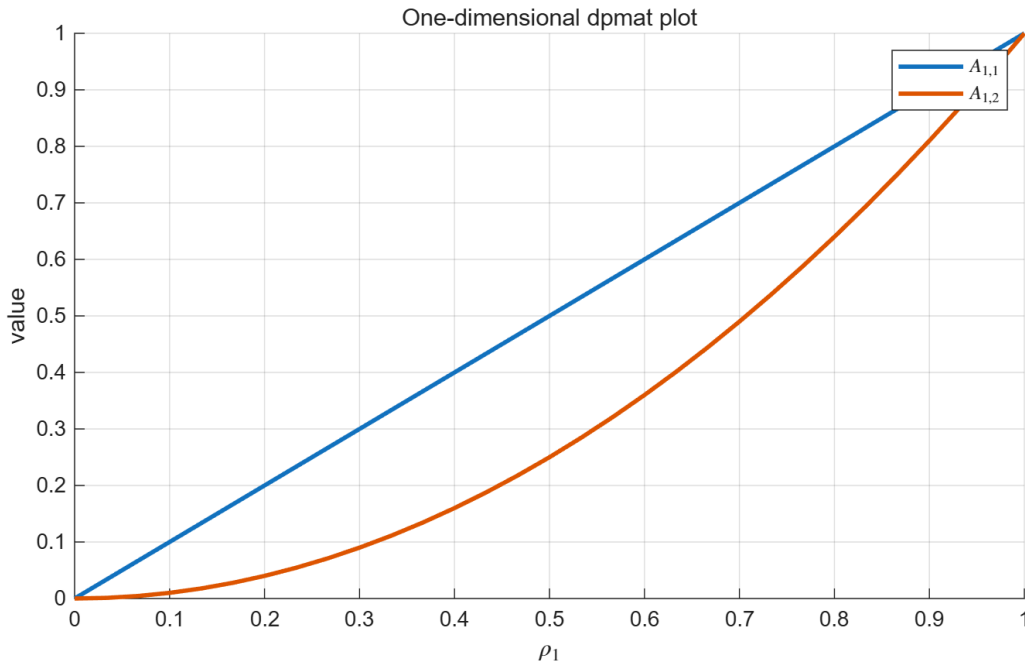


Figure 1: One-dimensional `dpmat` plot generated from MATLAB.

```
A = dpmat({[0 1], [10 20]}, @(rho, eta) [rho + eta, rho - eta]);
h = plot(A, [1 2], SamplesPerCell=2, EdgeColor="none");
numel(h)
```

Expected result: `numel(h)` is 2, one transparent surface for each matrix entry. Two-dimensional plotting defaults to `FaceAlpha=0.5` unless the caller supplies a different value, and accepts ordinary graphics options.

The figure files are generated by running `doc/generate_plot_figures.m` from the repository root. They live under `doc/matlab-figures/` so the manual source, generated plot assets, and regeneration entry point stay in the same documentation tree.

5.9 Validation Boundaries

`dpmat` rejects unsupported source types, malformed global coefficient grids, malformed nested local values, non-finite or complex payloads, bad function arity, empty or complex function outputs, non-polynomial handles when explicit Bernstein evidence is requested, and functions that require a higher `Degree` than the caller supplied. It also rejects constructor attempts to set `IsContinuous`, `ContainsDecision`, `HasRateDependence`, or `RateBounds`.

Operational errors include out-of-bounds or wrong-dimensional evaluation points, function-only coefficient algebra or structural transforms, incompatible matrix sizes, mixed physical grid bounds, `cat` dimensions other than 1 or 2, deletion assignment, matrix-growth assignment, block assignments with incompatible payload size, invalid reshape sizes, invalid diagonal or triangular offsets, and invalid plot dimensions or `SamplesPerCell`.

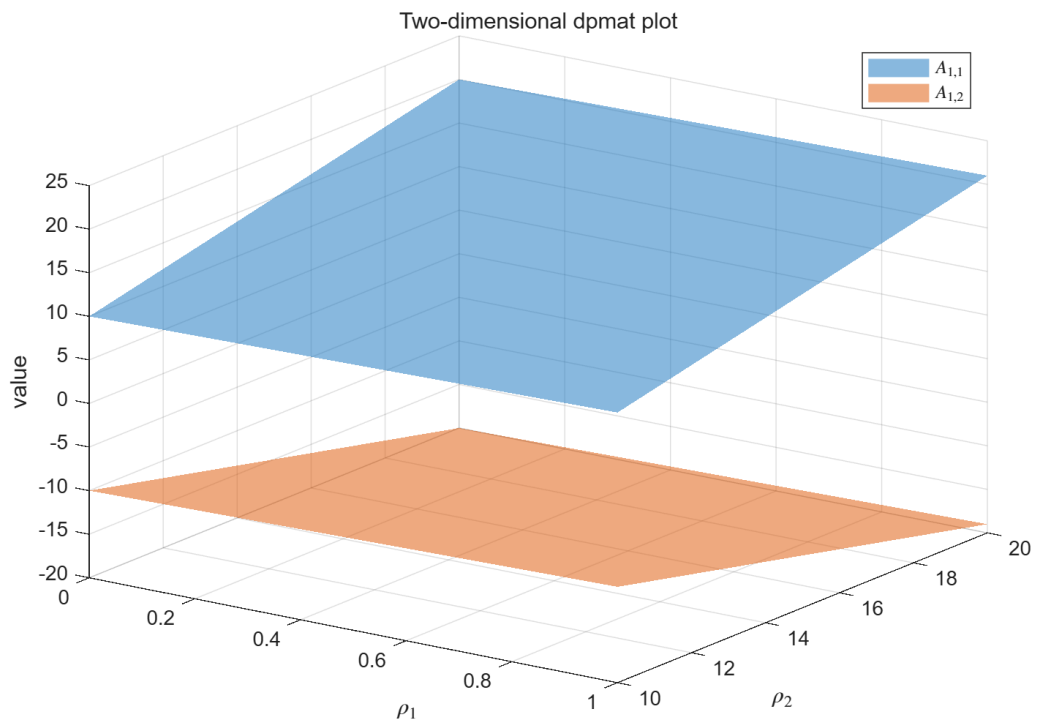


Figure 2: Two-dimensional `dpmat` surface plot generated from MATLAB.

6

dpvar Reference

6.1 Purpose

`dpvar` represents continuous degree-1 Bernstein decision variables on a tensor grid. Its coefficients are YALMIP `sdpvar` matrices stored through the same `dpbase` cell-local layout used by `dpmat`. The current slice is an affine expression layer. Comparisons such as $P \leq 0$ and $P \geq Q$ assemble `dplmi` objects, but `dpvar` itself does not choose solvers or call `optimize`.

6.2 Syntax

```
P = dpvar(n, gridVectors)
P = dpvar(n, n, gridVectors)
P = dpvar(n, m, gridVectors)
P = dpvar(..., "full")
P = dpvar(..., "symmetric")
P = dpvar(..., RateBounds=rb)
D = rhodiff(P, rb)
D = rhodiff(P)
C = P + Q
C = P - Q
C = -P
C = +P
C = P + A
C = A * P
C = P * A
T = P.'
H = P'
B = P(rows, cols)
P(rows, cols) = rhs
C = [P, A]
C = [P; P]
C = blkdiag(P, 1, A)
K = P <= 0
K = P >= Q
```

Here `A` may be coefficient-backed `dpmat` data or compatible numeric data. Bare affine `sdpvar` matrices can also be promoted to constant coefficient data in supported affine operations.

6.3 Construction And Continuity

```
P = dpvar(2, {[0 1 2]}, "symmetric");

size(P)
P.Degree
P.SourceSummary
first = P.coeffs(1);
```



```
second = P.coefts(2);
numel(getvariables(first{1}))
isequal(getvariables(first{2}), getvariables(second{1}))
```

Expected structural outputs:

```
size(P)      -> [2 2]
Degree      -> 1
SourceSummary -> "decision"
variables in first endpoint -> 3
shared boundary handle -> true
```

For a square matrix, `dpvar(n, gridVectors)` follows YALMIP's symmetric default. Use "full" when every matrix entry should be an independent decision variable. Rectangular matrices are full by construction.

Adjacent physical cells share the YALMIP coefficient handle at their common boundary. This is how the current constructor preserves continuity; it does not add equality constraints.

6.4 Rate Metadata

`RateBounds` can be supplied as metadata on a `dpvar`. The table has one row per parameter dimension and lower/upper bounds in columns one and two. The constructor stores the metadata outside `LocalValues`; it does not enumerate rate vertices.

```
rb = [-1 2; -3 4];
P = dpvar(1, {[0 1], [10 20]}, RateBounds=rb);

P.HasRateDependence
P.RateBounds
```

Expected output:

```
HasRateDependence -> true
RateBounds        -> [-1 2; -3 4]
```

6.5 Rate-Weighted Derivatives With `rhodiff`

`rhodiff` differentiates a `dpvar` with respect to the gridded parameters and multiplies the local partial derivatives by bounded $\dot{\rho}$ vertices. The output is another `dpvar`, but it is discontinuous across physical-cell boundaries because each cell keeps its own local derivative evidence. This is deliberate: derivative boundary data are cell-local and are not collapsed into a shared global node.

```
P = dpvar(1, {[0 2]}, RateBounds=[-2 3]);
D = rhodiff(P);
```

```
D.Degree
D.IsContinuous
D.HasRateDependence
D.RateBounds
size(D.coeffs(1))
```

Expected structural output:

```
Degree          -> 0
IsContinuous     -> false
HasRateDependence -> true
RateBounds       -> [-2 3]
size(D.coeffs(1)) -> [2 1]
```

For this scalar degree-one cell, if $c = P.coeffs(1)$, then the two stored derivative rows are

$$-2 \frac{c_1 - c_0}{2} \quad \text{and} \quad 3 \frac{c_1 - c_0}{2}.$$

The denominator is the physical cell width. In general, a scalar degree m cell $[\rho_i, \rho_{i+1}]$ stores

$$v \frac{m}{h_i} (c_{q+1} - c_q), \quad q = 0, \dots, m-1,$$

for each rate vertex v . Scalar derivatives of degree $m > 0$ therefore drop to degree $m-1$; degree-zero expressions differentiate to zero rows.

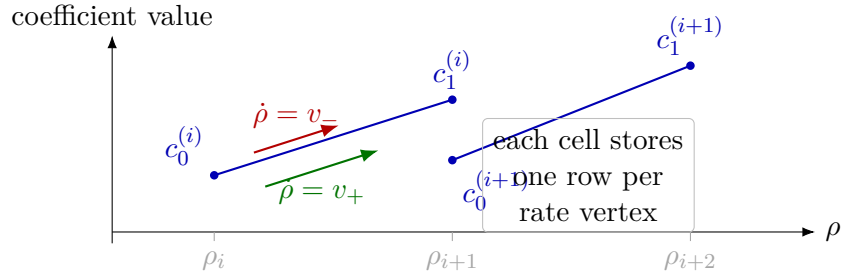


Figure 3: `rhodiff` keeps derivative evidence local to each physical cell. The cell slope is scaled once for each admissible rate vertex, so shared position coefficients need not imply shared derivative rows.

For tensor grids, `RateBounds` has one row per parameter dimension and `rhodiff` enumerates the Cartesian product of lower/upper bounds. A two-parameter grid with bounds $[-1, 2]$ and $[-3, 5]$ therefore stores four rate rows in the order

$$(-1, -3), \quad (-1, 5), \quad (2, -3), \quad (2, 5).$$

The local partial derivatives are accumulated after elevation into a common tensor Bernstein degree, so a degree-one two-parameter `dpvar` returns a degree-one derivative object with a `4x4` coefficient table on each physical cell.

The supported call forms are `rhodiff(P, rb)` and `rhodiff(P)`. The second form requires `P` to carry nonempty `RateBounds`. If explicit bounds are supplied and `P` already carries bounds, the two tables must match. Re-differentiating an existing rate-vertex expression is rejected in this slice.

6.6 Affine Algebra And Products

`dpvar` supports affine coefficient operations and products where at most one side contains decision variables. Products with coefficient-backed known data use the same Bernstein degree-growth rule as `dpmat`.

```
P = dpvar(1, {[0 1]});
A = dpmat({[0 1]}, {10, 20}, Degree=1);

L = A * P;
R = P * A;

L.Degree
R.Degree
L.ContainsDecision
```

Expected output:

```
L.Degree      -> 2
R.Degree      -> 2
ContainsDecision-> true
```

Same-bound mixed scalar grids are aligned on a common refinement. Mixed physical bounds are rejected. Products such as `P * Q`, `P * x` where `x` is a bare decision variable, and products involving rate metadata are rejected in this slice because they would leave the supported affine SDP expression layer.

Rate-vertex derivative expressions from `rhodiff` may still participate in supported affine operations. Ordinary `dpvar`, numeric, affine `sdpvar`, and coefficient-backed `dpmat` operands are broadcast across the derivative rows when the grids and matrix sizes are compatible. Products are allowed with numeric or coefficient-backed known data, but products between two rate-dependent expressions or between a rate-dependent expression and another decision expression are rejected.

6.7 Matrix Operations And Indexing

Matrix-like methods report and transform the YALMIP coefficient payloads, not the number of physical cells. The supported structural methods mirror the current `dpmat` coefficient-backed methods.

```
P = dpvar(2, {[0 1]}, "full");

length(P)
height(P)
width(P)
numel(P)
ndims(P)
size(vec(P))
```

```
size(diag(P))
size(reshape(P, [1 4]))
size(P(:, end))
```

Expected output:

```
length(P)      -> 2
height(P)      -> 2
width(P)       -> 2
numel(P)       -> 4
ndims(P)       -> 2
size(vec(P))   -> [4 1]
size(diag(P))  -> [2 1]
size(reshape(P, [1 4])) -> [1 4]
size(P(:,end)) -> [2 1]
```

Concatenation, `blkdiag`, structural transforms, matrix slicing, and matrix assignment accept numeric constants, affine `sdpvar` blocks, other compatible `dpvar` objects, and coefficient-backed `dpmat` blocks. They reject nonlinear YALMIP expressions, function-only `dpmat` operands, unsupported dimensions, matrix growth, deletion assignment, and incompatible rate metadata.

6.8 Validation Boundaries

`dpvar` constructor-created variables support `Degree=1` by default and `Degree=0` for parameter-independent decision expressions on the same grid metadata. Constructor-created variables are continuous decision objects; those flags are fixed internally. Attempts to pass the internal continuity, decision-content, or rate-dependence flags as constructor options are rejected. A `"symmetric"` structure flag requires a square matrix. Grid and `RateBounds` validation follows the shared `dpbase` rules.

7

dplmi Assembly And Solver-Facing Use

7.1 Purpose

`dplmi` wraps the finite YALMIP constraints produced from cell-local `dpvar` residuals. In the current implementation, `dpvar <= rhs` and `dpvar >= rhs` first form an affine residual using the supported `dpvar` algebra, then assemble one semidefinite constraint for every physical cell, every local Bernstein coefficient, and every active $\dot{\rho}$ rate-vertex row.

The current `dplmi` layer is deliberately direct: it constrains every stored Bernstein coefficient. Relaxation-lemma and PolyA options are accepted as named options only so unsupported requests can fail clearly.

7.2 Syntax

```
C = P <= Q
C = P <= 0
C = P >= Q
C = P >= eye(n)
C = dplmi(expr, "<=")
C = dplmi(expr, ">=", relaxLemma=false, UsePolya=false, PolyADegree=0)
F = toYalmip(C)
F = C.toYalmip()
```

Here `expr` must be a `dpvar` residual. The comparison overloads are usually the most convenient entry point because they reuse existing `dpvar` promotion rules for numeric constants, affine `sdpvar` matrices, coefficient-backed `dpmat` operands, and compatible `dpvar` objects.

7.3 Direct Coefficient Constraints

The simplest use is a coefficient-wise matrix inequality over a grid.

```
P = dpvar(2, {[0 1 2]}, "symmetric");
Cpos = P >= eye(2);

class(Cpos)
numel(Cpos.Constraints)
Cpos.RelaxLemma
Cpos.UsePolya
Cpos.PolyADegree
```

Expected structural output:

```
class(Cpos)          -> "dplmi"
numel(Cpos.Constraints) -> 4
RelaxLemma           -> false
UsePolya              -> false
PolyADegree           -> 0
```

There are two physical cells and two degree-one local coefficients per cell, so direct assembly creates four scalar YALMIP semidefinite constraint entries. The entries remain inspectable in `Cpos.Constraints`; use `toYalmip(Cpos)` only at the solver boundary.

7.4 Rate-Vertex Assembly From `rhodiff`

When the residual contains `rhodiff(P, rb)`, each local coefficient table has one row per admissible $\dot{\rho}$ vertex. `dplmi` constrains every row.

```
P = dpvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
Cdecay = D <= 0;

D.Degree
size(D.coeffs(1))
numel(Cdecay.Constraints)
```

Expected structural output:

```
D.Degree          -> 0
size(D.coeffs(1)) -> [2 1]
numel(Cdecay.Constraints)-> 4
```

The grid has two physical cells. The scalar rate interval $[-1, 1]$ has two vertices. The degree-one derivative drops to degree zero, so each cell has one local derivative coefficient per rate vertex: 2 cells times 2 rate rows.

For a two-parameter grid, the same rule applies to the tensor product of rate vertices:

```
rb = [-1 1; -2 2];
P = dpvar(2, {[0 1], [0 1]}, "symmetric", RateBounds=rb);
D = rhodiff(P);
C = D <= 0;

size(D.coeffs([1 1]))
numel(C.Constraints)
```

Expected structural output:

```
size(D.coeffs([1 1])) -> [4 4]
numel(C.Constraints) -> 16
```

There is one physical cell, four rate vertices, and four degree-one tensor coefficients after multivariate partial derivatives are elevated into a common tensor basis.

7.5 A Lyapunov-Style Solver Smoke Test

The following example mirrors the current solver smoke test. It compares a constant decision matrix against a parameter-dependent one for the same coefficient-wise decay inequality.

```
A = dpmat({[0 1]}, @(x) (1 - x) * [-1 -1; 1 -1] ...
    + x * [-1 -10; 0.1 -1], Degree=1);
opts = sdpsettings('verbose', 0);

Pc = dpvar(2, {[0 1]}, Degree=0);
Cdecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
```

```

Cpos = Pc >= eye(2);
solConst = optimize([Cdecay.toYalmip, Cpos.toYalmip], [], opts);

Pd = dpvar(2, {[0 1]});
Cdecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
Cpos = Pd >= eye(2);
solDp = optimize([Cdecay.toYalmip, Cpos.toYalmip], [], opts);

[solConst.problem, solDp.problem]

```

Expected result under an available SDP solver:

```
[solConst.problem, solDp.problem] -> [1 0]
```

YALMIP returns **problem=1** for the constant- P infeasible case and **problem=0** for the degree-one parameter-dependent feasible case in the current local test setup.

7.6 A Block DP-LMI With A Performance Variable

This example is adapted from the root `test.m` case. The test suite now keeps it in the `dplmi` solver-smoke tests. It combines known parameter-dependent matrices, a rate-weighted derivative, a scalar degree-zero performance variable, block matrix assembly, and MOSEK-backed optimization.

```

grid = linspace(0, 1, 20);
A = dpmat(grid, @(x) [-1, 0.5; -1, -2] ...
    + x * [-1.3, -20; 2, -10], Degree=1);
B = dpmat(grid, @(x) [1, -4; -1, -1] ...
    + x * [2.2, 0.5; -6, -5], Degree=1);
C = eye(2);
D = zeros(2);

P = dpvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = dpvar(1, grid, Degree=0);

E1 = [diffP + P * A + A' * P, P * B, C';
      B' * P, -gamma * eye(2), D';
      C, D, -gamma * eye(2)] <= 0;
E2 = P >= 0;

numel(E1.Constraints)
numel(E2.Constraints)

opts = sdpsettings('solver', 'mosek', 'verbose', 0);
objective = gamma.LocalValues{1}{1};
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
value(objective)

```

Expected structural output before the solver call:

```

numel(E1.Constraints) -> 114
numel(E2.Constraints) -> 38

```

The first block inequality has 19 physical intervals, degree-two residual coefficients, and two rate vertices from `diffP`, so it produces $19 \times 3 \times 2 = 114$ direct constraints. The positivity constraint

has 19 intervals and two degree-one coefficients, so it produces $19 \times 2 = 38$ constraints. The final numeric value depends on the installed solver and tolerances; the test verifies a successful MOSEK solve and a finite objective value when MOSEK is available.

7.7 Validation Boundaries

`dplmi` requires square symmetric or Hermitian coefficient matrices. Non-square residuals fail with `dplmi:InvalidMatrixSize`; non-symmetric residuals fail with `dplmi:NonSymmetricExpression`. Unsupported relaxation and PolyA requests fail explicitly:

```
P = dpvar(2, {[0 1]}, "symmetric");

dplmi(P, "<=", relaxLemma=true) % dplmi:UnsupportedRelaxLemma
dplmi(P, "<=", UsePolya=true)   % dplmi:UnsupportedPolya
dplmi(P, "<=", PolyADegree=1)   % dplmi:UnsupportedPolya
```

Direct coefficient-wise assembly is the only implemented route in this slice. Strictness margins, theorem residual margins, relaxation slacks, PolyA lifting, diagnostics, and high-level solver wrappers remain future work.

8

Current Limits

This repository version intentionally has a narrow public surface.

- The backend parent `dpbase`, user-facing `dpmat` and `dpvar` classes, `helper.chk`, `helper.scanMats`, and `helper.cell2bern` are implemented and covered by the current MATLAB test entry point.
- `dpmat` is a known-data object only. It does not contain YALMIP decisions, rate dependence, exact function-handle algebra beyond evaluation/plot sampling, or solver-facing LMI assembly.
- `dpvar` is currently a continuous degree-1 YALMIP-backed affine expression object with `rhodiff` support for discontinuous cell-local rate-vertex derivatives. It does not implement nonlinear decision products, products with rate dependence on both sides, or solver calls.
- `dplmi` implements direct coefficient-wise finite YALMIP constraint assembly from `dpvar` residuals. It does not yet implement relaxation-lemma assembly, Polya assembly, strictness margins, residual evidence, diagnostics, or high-level solver wrappers.
- There is no public `dpdiff` class. Public derivative behavior belongs to `dpvar` through `rhodiff`.
- YALMIP solver invocation is available through ordinary YALMIP calls after converting `dplmi` objects with `toYalmip`; this package does not yet provide a custom solver wrapper.
- Private Bernstein utilities support internal tests and implementation, but they are not user-facing functions.

9

Style References Used For This Manual

This manual follows the organization style common in mature MATLAB and control-toolbox documentation: installation first, a small runnable example, conceptual background, then API reference with syntax, arguments, examples, validation rules, and current limits. Local LPVTools, ROLMIP, and ROMULOC documentation were used as style references only. Their APIs and text are not dependencies of this package and are not copied here.